



YACHAY TECH UNIVERSITY

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

LABORATORY 4 REPORT

RSA Factorization Attacks

Trial Division, Fermat and Pollard's Rho against small RSA moduli

AUTHOR Daniel Troya A.

COURSE Cryptography

INSTRUCTOR PhD. Christian Iza

URCUQUÍ, ECUADOR

September 2026

1 Introduction

The security of RSA rests on one assumption: given $n = pq$, finding p and q is hard [1]. This report tests that assumption on small moduli. Three classic factorization algorithms (Trial Division, Fermat and Pollard's Rho) were implemented in Python from scratch, used to recover an RSA private key and decrypt a ciphertext, and then compared on moduli of 16 to 48 bits and on moduli with different distances $|p - q|$.

2 Objectives

2.1 General Objective

Implement and evaluate three factorization algorithms against small RSA moduli, and show that factoring n is enough to recover the private key.

2.2 Specific Objectives

- Implement the three algorithms without libraries that factor n .
- Recover the private key of $(n, e) = (3233, 17)$ and decrypt $c = 2790$.
- Measure the algorithms on moduli of increasing size and of different $|p - q|$.

3 Background

3.1 RSA and factorization

An RSA key is built from two primes p and q [2]:

$$n = pq, \quad \varphi(n) = (p - 1)(q - 1), \quad d \equiv e^{-1} \pmod{\varphi(n)}, \quad (1)$$

with public key (n, e) and private key (n, d) . Encryption is $c = m^e \bmod n$ and decryption is $m = c^d \bmod n$. The public key reveals n but not $\varphi(n)$, and computing $\varphi(n)$ from n alone is as hard as factoring n . Once p and q are known, equation (1) gives d directly, so an attacker who factors n holds the private key.

3.2 Factorization algorithms

Trial Division tests the candidates $2, 3, 5, 7, 9, \dots$ up to $\sqrt{n}$. For an RSA modulus the smallest factor is close to $\sqrt{n}$, so it needs about $\sqrt{n}/2$ divisions, that is $O(\sqrt{n})$.

Fermat's method writes n as a difference of squares, $n = a^2 - b^2 = (a - b)(a + b)$. It starts at $a = \lceil \sqrt{n} \rceil$ and increases a until $a^2 - n$ is a perfect square b^2 . The search stops at $a = (p + q)/2$,

so the exact number of iterations is

$$I_F = \frac{p+q}{2} - \lceil \sqrt{n} \rceil + 1 \approx \frac{(q-p)^2}{8\sqrt{n}}. \quad (2)$$

The cost depends on the distance between the factors, not on the size of n .

Pollard's Rho iterates $f(x) = x^2 + c \bmod n$. Modulo the unknown factor p , the sequence must repeat, and by the birthday paradox it does so after about $\sqrt{p}$ steps. Two pointers move through the sequence, x one step and y two steps at a time (Floyd's cycle detection), and each step computes $d = \gcd(|x - y|, n)$. When $x \equiv y \pmod{p}$, p divides $x - y$ and therefore d . The expected cost is $O(\sqrt{p}) = O(n^{1/4})$ [3].

4 Implementation

4.1 Software design

The code keeps factorization, key recovery, tests and experiments separate. `src/number_theory.py` holds gcd, extended Euclid, modular inverse, modular exponentiation, integer square root and Miller-Rabin; `src/factorization.py` holds the three algorithms, each returning p , q and its number of iterations; `src/rsa.py` handles key generation, encryption, decryption and key recovery. The `tests/` folder holds the automatic tests, and `experiments/` the benchmark, the Fermat analysis and the plots, with results stored as JSON. The only external libraries are `pytest` and `matplotlib`.

4.2 Relevant decisions

The integer square root uses Newton's method on integers only, so it stays exact for any size of n . Before calling it, the perfect-square test rejects most candidates with one check, since a square modulo 16 can only be 0, 1, 4 or 9. Fermat updates b^2 with $(a+1)^2 - n = (a^2 - n) + 2a + 1$ instead of squaring a again. Pollard's Rho starts with $c = 1$ and $x_0 = 2$; if $d = n$, the cycle closed modulo n and it restarts with $c + 1$.

Test primes come from Miller-Rabin with the first twelve primes as bases (deterministic below 3.3×10^{24}), with equal bit sizes so n has exactly the requested size. Every algorithm also counts its iterations, a cost measure that does not depend on the computer.

4.3 Validation

The test suite has 92 tests and all of them pass (table 1).

Table 1. Test suite coverage (92/92 passing).

Group	Tests	What it checks
GCD	8	Known values and the Bézout identity $as + bt = \gcd(a, b)$.
Modular inverse	4	$17^{-1} \bmod 3120 = 2753$; error if $\gcd(a, m) \neq 1$.
Integer square root	11	$k^2 \leq n < (k+1)^2$ on 200 random 200-bit numbers.
Primes, modular power	9	Miller-Rabin against a sieve; <code>mod_pow</code> against <code>pow</code> .
Factorization	46	Known and random moduli, $pq = n$, prime inputs rejected.
RSA, result files	14	Lab key, $\text{Dec}(\text{Enc}(m)) = m$, exact JSON storage.

5 Experimental Setup

All runs used the same machine: an Intel Core Ultra 7 270K Plus (24 cores, 3.70 GHz) with 32 GB of RAM, Windows 11 Pro and CPython 3.12.10. Each modulus was factored three times per algorithm, timing only the factorization with `time.perf_counter`, and every run checked $pq = n$. The benchmark moduli come from a fixed seed (2026), so the experiment can be repeated with the same values.

6 Results

6.1 Private key recovery

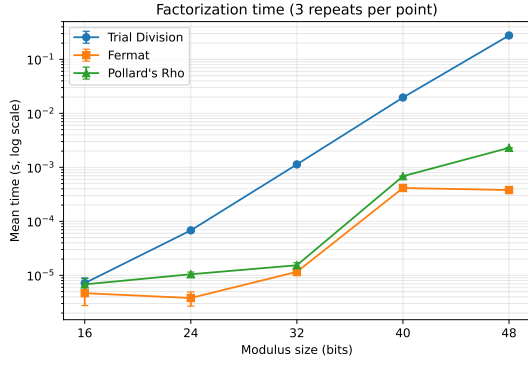
The three algorithms factor $n = 3233$ into $p = 53$ and $q = 61$, the values expected by the lab. From there, $\varphi(n) = 52 \cdot 60 = 3120$ and $d = 17^{-1} \bmod 3120 = 2753$, since $17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1$. The private key is $(n, d) = (3233, 2753)$, the decrypted message is $m = 2790^{2753} \bmod 3233 = 65$, and $65^{17} \bmod 3233 = 2790$ confirms it. Trial Division needed 27 iterations, Fermat 1 and Pollard’s Rho 6, all in under $15 \mu\text{s}$.

6.2 Benchmark

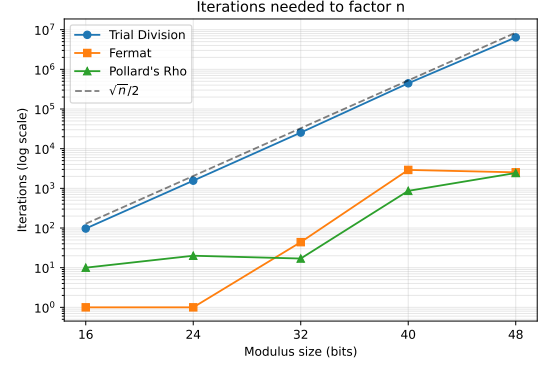
Table 2 and figure 1 give the average time and the iterations for five modulus sizes.

Table 2. Average factorization time in ms (3 runs); iterations in parentheses.

Bits	n	Trial Division	Fermat	Pollard’s Rho
16	43 039	0.0072 (97)	0.0047 (1)	0.0068 (10)
24	10 015 289	0.0680 (1 561)	0.0038 (1)	0.0105 (20)
32	2 809 649 851	1.1332 (25 447)	0.0116 (44)	0.0154 (17)
40	923 962 313 633	19.6286 (444 624)	0.4141 (2 914)	0.6846 (864)
48	169 783 438 229 137	276.6169 (6 387 832)	0.3798 (2 534)	2.2953 (2 437)



(a) Average time.

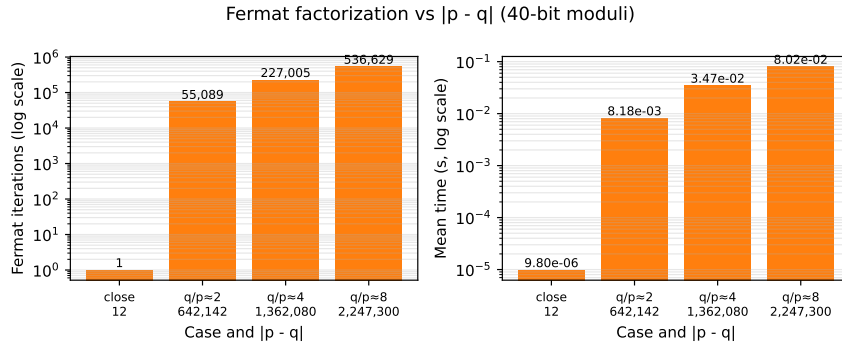
(b) Iterations, with the reference $\sqrt{n}/2$.**Figure 1.** Cost of the three algorithms against the modulus size (logarithmic scale).

6.3 Fermat and the distance between p and q

Four 40-bit moduli were built around the same value, changing only the ratio q/p . Table 3 and figure 2 show the results. The measured iterations match equation (2) exactly in the four cases.

Table 3. Fermat on 40-bit moduli with different $|p - q|$ (3 runs).

Case	p	q	$ p - q $	Iterations	Time (ms)
Close	908 101	908 113	12	1	0.0098
Far, $q/p \approx 2$	642 121	1 284 263	642 142	55 089	8.179
Far, $q/p \approx 4$	454 061	1 816 141	1 362 080	227 005	34.687
Far, $q/p \approx 8$	321 073	2 568 373	2 247 300	536 629	80.154

**Figure 2.** Fermat iterations and time for each $|p - q|$ (logarithmic scale).

7 Analysis

How does the size of n affect Trial Division? Its cost grows with $\sqrt{n}$: every 8 extra bits multiply $\sqrt{n}$ by 16. The measurements follow this, with the iterations growing by 16.1, 16.3, 17.5 and 14.4 between consecutive sizes, and the curve in figure 1 staying on $\sqrt{n}/2$. At 48 bits it needed 6.4 million divisions and 277 ms, so it is useless against real RSA moduli.

Why is Fermat efficient when p and q are close? The search starts at $\sqrt{n}$ and ends at $(p + q)/2$, and these two values are almost equal when $p \approx q$. By equation (2), the cost grows with $(q - p)^2$ and not with n . Table 3 shows it: at the same size, a distance of 12 needs one iteration, while a distance of 2.2 million needs 536 629 iterations and 8 000 times more time.

What role does the GCD play in Pollard’s Rho? It detects a collision modulo p without knowing p . The values x and y are different modulo n , but once they agree modulo p , the difference $x - y$ is a multiple of p and $\gcd(|x - y|, n)$ returns it. The GCD is also the most expensive step: Pollard’s Rho costs about $0.9 \mu\text{s}$ per iteration, against $0.15 \mu\text{s}$ for Fermat and $0.04 \mu\text{s}$ for Trial Division. Its advantage comes from needing few iterations, about $\sqrt{p}$, which is 2 437 at 48 bits.

Which algorithm performed better for the largest modulus? At 48 bits, Fermat was the fastest (0.38 ms), followed by Pollard’s Rho (2.30 ms) and Trial Division (276.6 ms). Fermat wins only because the test primes have the same bit size and are relatively close: with $q/p \approx 8$ at 40 bits it needed 80.2 ms, over 100 times more than Pollard’s Rho on the 40-bit benchmark modulus. Pollard’s Rho depends only on $\sqrt{p}$, so it is the most reliable choice when the structure of n is unknown.

Why does recovering p and q break the RSA private key? The only secret needed to compute d is $\varphi(n) = (p - 1)(q - 1)$. With p and q , the attacker repeats the key owner’s computation, and the extended Euclidean algorithm returns d at negligible cost, as equation (1) and the lab key show.

8 Conclusions

- All three algorithms recover the key (3233, 2753) and $m = 65$.
- Trial Division grows with $\sqrt{n}$: 16 times more work every 8 bits.
- Fermat depends on $|p - q|$, not on the size of n , and its iterations match equation (2) exactly. Pollard’s Rho grows with $\sqrt{p}$ whatever the factors are, so it scales best in general.
- RSA is secure only when n is too large to factor and p and q are not too close.

References

- [1] R. L. Rivest, A. Shamir, and L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.
- [2] C. Paar and J. Pelzl, *Understanding Cryptography: A Textbook for Students and Practitioners*. Berlin, Germany: Springer-Verlag, 2010.
- [3] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2020.